

Lesson 8

面向对象的设计原则

主讲老师：高祥



2024/10/31

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

- 面向对象设计的原则
- 对象的equals()方法与操作符“==”
- 内部类
- 匿名类
- 内部类的用途

设计原则的必要性

- 了解面向对象设计的基本原则，有助于知道如何合理使用面向对象语言：
 - 高内聚
 - 低耦合
 - 易维护
 - 易扩展
 - 易复用

高内聚-低耦合原则

- 耦合主要描述模块之间的关系
- 内聚主要描述模块内部

低耦合

- **低耦合**：是指软件系统中，模块与模块之间的直接依赖程度低。
- 模块之间存在依赖,模块独立性越差，耦合越强,导致一个模块的改动可能会影响到其他模块。**比如模块A直接操作了模块B中数据,则视为强耦合,若A只是通过数据与模块B交互,则视为弱耦合。**
- **好处**：独立的模块便于扩展,维护,写单元测试,如果模块之间重重依赖,会极大降低开发效率，代码可维护性差。

高内聚

- **高内聚**：系统存在AB两个模块儿进行交互，如果修改了A模块儿不影响B模块的工作，那么认为A模块儿有足够的内聚。
- 一个模块应当尽可能独立完成某个功能。模块内部的元素，关联性越强，则内聚越高，模块单一性更强。
- **危害**：低内聚的模块代码，不管是维护，扩展还是重构都相当麻烦，难以下手。

面向对象的基本原则

设计原则	一句话归纳	目的
开闭原则	对扩展开放，对修改关闭	降低维护带来的新风险
依赖倒置原则	高层不应该依赖低层，要面向接口编程	更利于代码结构的升级扩展
单一职责原则	一个类只干一件事，实现类要单一	便于理解，提高代码的可读性
接口隔离原则	一个接口只干一件事，接口要精简单一	功能解耦，高聚合、低耦合
迪米特法则	不该知道的不要知道，一个类应该保持对其它对象最少的了解，降低耦合度	只和朋友交流，不和陌生人说话，减少代码臃肿
里氏替换原则	不要破坏继承体系，子类重写方法功能发生改变，不应该影响父类方法的含义	防止继承泛滥
合成复用原则	尽量使用组合或者聚合关系实现代码复用，少使用继承	降低代码耦合

开闭原则

- 开闭原则: 一个软件实体应当对扩展开放, 对修改关闭。
- 当软件需要变化时, 尽量通过扩展软件实体的行为来实现变化, 而不是通过修改已有的代码来实现变化
- 可扩展性

开-闭原则

- 所谓 开-闭原则（Open-Closed Principle）就是让你的设计应当对扩展开放，对修改关闭。
- 在给出一个设计时，应当首先考虑到**用户需求的变化**，将应对用户变化的部分设计为对扩展开放，而设计的核心部分是经过精心考虑之后确定下来的基本结构，这部分应当是对修改关闭的，即不能因为用户的需求变化而再发生变化，因为这部分不是用来应对需求变化的

经常需要**面向抽象**来考虑系统的总体设计，不要考虑具体类，这样就容易设计出满足“开-闭原则”的系统。

违背开-闭原则

实现画图类，绘制不同图形：

```
class GraphicEditor {  
    //接收Shape对象，然后根据type，来绘制不同的图形  
    public void drawShape(Shape s) {  
        if (s.m_type == 1) {  
            drawRectangle(s);  
        } else if (s.m_type == 2) {  
            drawCircle(s);  
        } else if (s.m_type == 3) {  
            drawTriangle(s);  
        }  
    }  
    //绘制矩形  
    public void drawRectangle(Shape r){  
        System.out.println("绘制矩形");  
    }  
    //绘制圆形  
    public void drawCircle(Shape r){  
        System.out.println("绘制圆形");  
    }  
    //绘制三角形  
    public void drawTriangle(Shape r){  
        System.out.println("绘制三角形");  
    }  
}
```

```
class Shape{  
    int m_type;  
}  
class Rectangle extends Shape{  
    Rectangle(){  
        super.m_type=1;  
    }  
}  
class Circle extends Shape{  
    Circle(){  
        super.m_type=2;  
    }  
}  
class Triangle extends Shape{  
    Triangle(){  
        super.m_type=3;  
    }  
}
```

增加其他形状？

开-闭原则

实现画图类，绘制不同图形：

```
abstract class Shape{
    int m_type;
    //抽象方法
    public abstract void draw();
}
//这是一个用于绘图类[使用方]
class GraphicEditor {
    //接收Shape对象，调用draw方法
    public void drawShape(Shape s) {
        s.draw();
    }
}
```

面向抽象来考虑系统的总体设计

闭

```
class Rectangle extends Shape {
    Rectangle(){
        super.m_type=1;
    }
    @Override
    public void draw() {
        System.out.println("绘制矩形");
    }
}
class Circle extends Shape {
    Circle(){
        super.m_type=2;
    }
    @Override
    public void draw() {
        System.out.println("绘制圆形");
    }
}
class Triangle extends Shape {
    Triangle(){
        super.m_type=3;
    }
    @Override
    public void draw() {
        System.out.println("绘制三角形");
    }
}
```

开

单一职责原则

- **单一职责原则**: 一个类只负责一个功能领域中的相应职责。
- 在程序设计时, 我们应该将程序功能最小化, 每个函数只干一件事。

里氏代换原则

- **里氏代换原则**: 所有引用基类（父类）的地方必须能透明地使用其子类的对象。
- 任何基类可以出现的地方，子类一定可以出现。
- 里氏代换原则是继承复用的基石，只有当衍生类可以替换掉基类，软件单位的功能不受到影响时，基类才能真正被复用
- 子类继承父类时，除添加新的方法完成新增功能外，尽量不要重写父类的方法
- 实现“开-闭”原则的关键步骤就是抽象化。里氏代换原则是对实现抽象化的具体步骤的规范。

违背里氏代换原则

//长方形

```
public class Changfangxing{
    private long width;
    private long height;

    public long getWidth() {
        return width;
    }
    public void setWidth(long width) {
        this.width = width;
    }
    public long getHeight() {
        return height;
    }
    public void setHeight(long height) {
        this.height = height;
    }
}
```

//正方形

```
public class Zhengfangxing{
    private long side;

    public long getSide() {
        return side;
    }

    public void setSide(long side) {
        this.side = side;
    }
}
```

//正方形

```
public class Zhengfangxing extends  
Changfangxing{  
    private long side;  
  
    public long getHeight() {  
        return this.getSide();  
    }  
  
    public long getWidth() {  
        return this.getSide();  
    }  
}
```

```
    public void setHeight(long height) {  
        this.setSide(height);  
    }  
  
    public void setWidth(long width) {  
        this.setSide(width);  
    }  
  
    public long getSide() {  
        return side;  
    }  
  
    public void setSide(long side) {  
        this.side = side;  
    }  
}
```

违背里氏代换原则

```
public class MainClass {  
    public static void main(String[] args) {  
        Changfangxing changfangxing = new Changfangxing();  
        changfangxing.setHeight(10);  
        changfangxing.setWidth(20);  
        resize(changfangxing);  
  
        Changfangxing zhengfangxing = new Zhengfangxing();  
        zhengfangxing.setHeight(10);  
        resize(zhengfangxing);  
    }  
  
    public static void resize(Changfangxing changfangxing) {  
        while (changfangxing.getHeight() >= changfangxing.getWidth()) {  
            int temp = changfangxing.getHeight();  
            changfangxing.setWidth(changfangxing.getHeight());  
            changfangxing.setHeight(temp);  
        }  
    }  
}
```

基类可以出现的地方，子类一定可以出现？死循环

合成复用原则

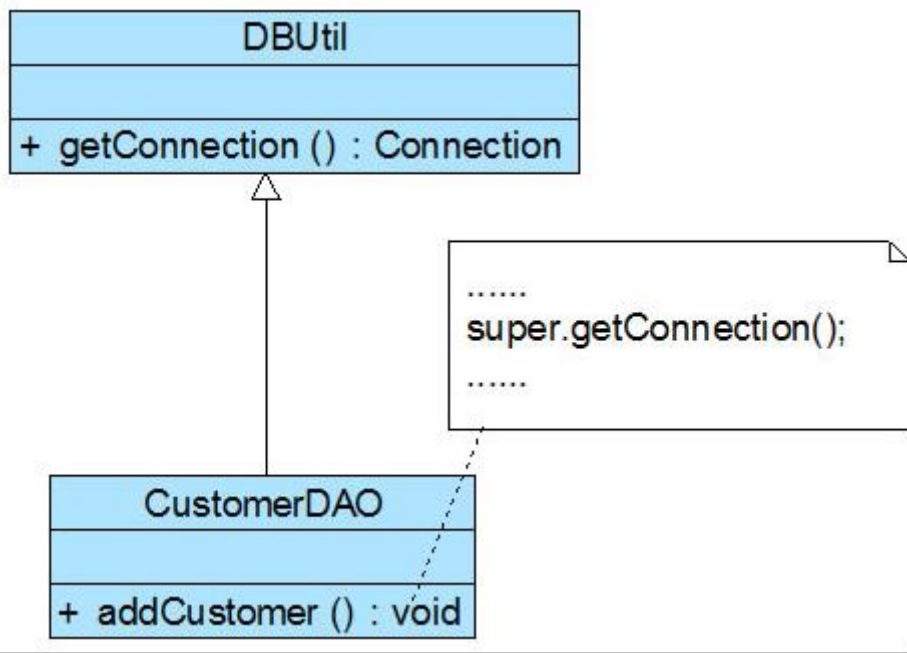
- 合成复用原则：尽量使用组合或者聚合关系实现代码复用，少使用继承。
- 如果要使用继承关系，则必须严格遵循里氏替换原则。合成复用原则同里氏替换原则相辅相成的，两者都是开闭原则的具体实现规范。

优先使用组合少用继承原则

- 方法复用的两种最常用的技术就是**类继承**和**对象组合**
- 通过继承复用方法的缺点是：
 - 继承破坏了封装性，通过继承进行复用也称“**白盒**”**复用**，其缺点是父类的内部细节对于子类而言是可见的。
 - 子类和父类的关系是**强耦合关系**，也就是说当父类的方法的行为更改时，必然导致子类发生变化
 - 子类从父类继承的方法在编译时刻就确定下来了，所以**无法在运行期间改变从父类继承的方法的行为**。

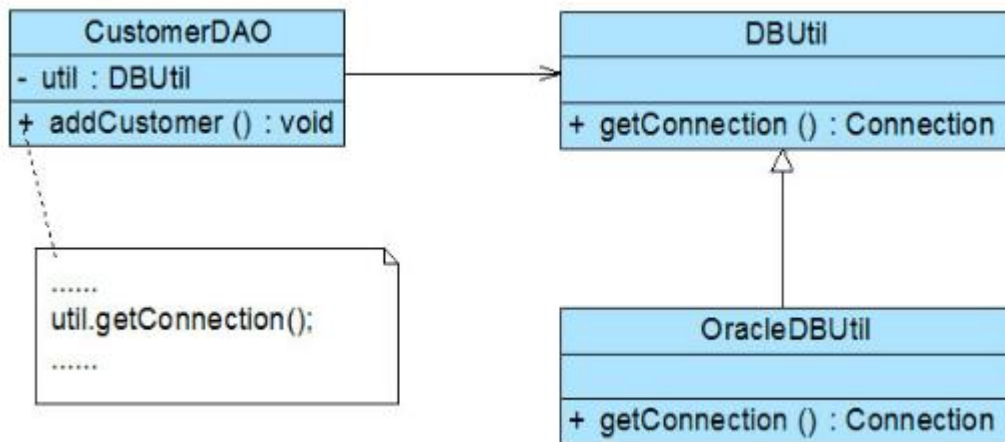
违反合成复用原则

- 开发人员在初期考虑到客户数量不多，系统采用MySQL作为数据库，
- 连接数据库的方法getConnection()封装在DBUtil类中
- 由于需要重用DBUtil类的getConnection()方法，设计人员将CustomerDAO作为DBUtil类的子类



- 随着客户数量的增加，系统决定升级为Oracle数据库
- 需要增加一个新的OracleDBUtil类来连接Oracle数据库
- 由于CustomerDAO和DBUtil之间是继承关系，在更换数据库连接方式时需要将CustomerDAO作为OracleDBUtil的子类
- 违反开闭原则。

合成复用原则正确性示范



- CustomerDAO和DBUtil之间的关系由继承关系变为关联关系
- CustomerDAO持有DBUtil的对象（util.getConnection()）
- 如果需要对DBUtil的功能进行扩展，可以通过其子类来实现，如通过子类OracleDBUtil来连接Oracle数据库
- 根据里氏代换原则，DBUtil子类的对象可以覆盖DBUtil对象，只需在CustomerDAO中注入子类对象即可使用子类所扩展的方法

组合与复用

- 组合对象来复用方法的优点是：
 - 对象组合是类继承之外的另一种复用选择。新的更复杂的功能可以通过组合对象来获得。
 - 对象组合要求对象具有良好定义的接口。这种复用风格被称为**黑箱复用(black-box reuse)**，因为被组合的对象的内部细节是不可见的,对象只以"**黑箱**"的形式出现。
- **对象与所包含的对象属于弱耦合关系**，因为，如果修改当前对象所包含的对象的类的代码，不必修改当前对象的类的代码。
- 当前对象可以在**运行时刻动态指定所包含的对象**。

多用组合少用继承原则

- 之所以提倡多用组合，少用继承，是因为在许多设计中，人们希望系统的类之间尽量是低耦合的关系，而不希望是强耦合关系。即在许多情况下需要避开继承的缺点，而需要组合的优点。
- 怎样合理地使用组合，而不是使用继承来获得方法的复用需要经过一定时间的认真思考、学习和编程实践才能悟出其中的道理。

依赖倒转原则

- **依赖倒转原则**: 抽象不应该依赖于细节, 细节应当依赖于抽象。换言之, 要针对接口编程, 而不是针对实现编程。
- 通过抽象使各个类或模块的实现彼此独立, 不互相影响, 实现模块间的松耦合
- 面向接口、抽象类编程

面向接口的编程

- **面向接口的编程**：是面向对象设计（OOD）的非常重要的基本原则之一。
- 它的含义是：使用接口和同类型的组件通讯，即，对于所有完成相同功能的组件，应该抽象出一个接口，它们都实现该接口。

面向抽象原则

- 当设计一个类时，不让该类面向具体的类，而是面向抽象类或接口，即所设计类中的重要数据是抽象类或接口声明的变量，而不是具体类声明的变量
- 具体到JAVA中，可以是接口（interface），或者是抽象类（abstract class），所有完成相同功能的组件都实现该接口，或者从该抽象类继承。
- 客户代码只应该和该接口通讯。

面向抽象原则（抽象类和接口）

案例：

已有Circle类，该类创建的对象circle调用getArea()方法可以计算圆的面积。

现在要设计一个Pillar类（柱类），该类的对象调用getVolume()方法可以计算柱体的体积

```
public class Pillar {  
    Circle bottom; //将Circle对象作为成员，bottom是用具体类声明的变量  
    double height;  
    Pillar (Circle bottom,double height) {  
        this.bottom=bottom;this.height=height;  
    }  
    public double getVolume() {  
        return bottom.getArea()*height;  
    }  
}
```

面向抽象编程的思想

- 如果不涉及用户需求的变化，上面Pillar类的设计没有什么不妥，但是在某个时候，用户希望Pillar类能创建出底是三角形的柱体。显然上述Pillar类无法创建出这样的柱体，即上述设计的Pillar类不能应对用户的这种需求。
- 因此，我们在设计Pillar类时**不应当让它的底是某个具体类声明的变量**，一旦这样做，Pillar类就依赖该具体类，缺乏弹性，难以应对需求的变化。

面向抽象编程的思想

- 首先编写一个抽象类Geometry（或接口），该抽象类（接口）中定义了一个抽象的getArea()方法

```
public abstract class Geometry { //如果使用接口需用interface来定义Geometry。
    public abstract double getArea();
}
```

```
public class Pillar {
    Geometry bottom; //bottom是抽象类Geometry声明的变量
    double height;
    Pillar (Geometry bottom,double height) {
        this.bottom = bottom; this.height=height;
    }
    public double getVolume() {
        return bottom.getArea()*height;
    }
}
```

Pillar类的设计
不再依赖具体
类，而是面向
Geometry类

面向抽象编程的思想

```
public class Circle extends Geometry
{
    double r;
    Circle(double r) {
        this.r=r;
    }
    public double getArea() {
        return(3.14*r*r);
    }
}
```

```
public class Rectangle extends Geometry {
    double a,b;
    Lader(double a,double b) {
        this.a=a; this.b=b;
    }
    public double getArea() {
        return a*b;
    }
}
```

```
public class Application{
    public static void main(String args[]){
        Pillar pillar;
        Geometry bottom;
        bottom=new Rectangle(12,22,100);
        pillar = new Pillar (bottom,58); //pillar是具有矩形底的柱体
        System.out.println("矩形底的柱体的体积"+pillar.getVolume());
        bottom=new Circle(10);
        pillar = new Pillar (bottom,58); //pillar是具有圆形底的柱体
        System.out.println("圆形底的柱体的体积"+pillar.getVolume());
    }
}
```

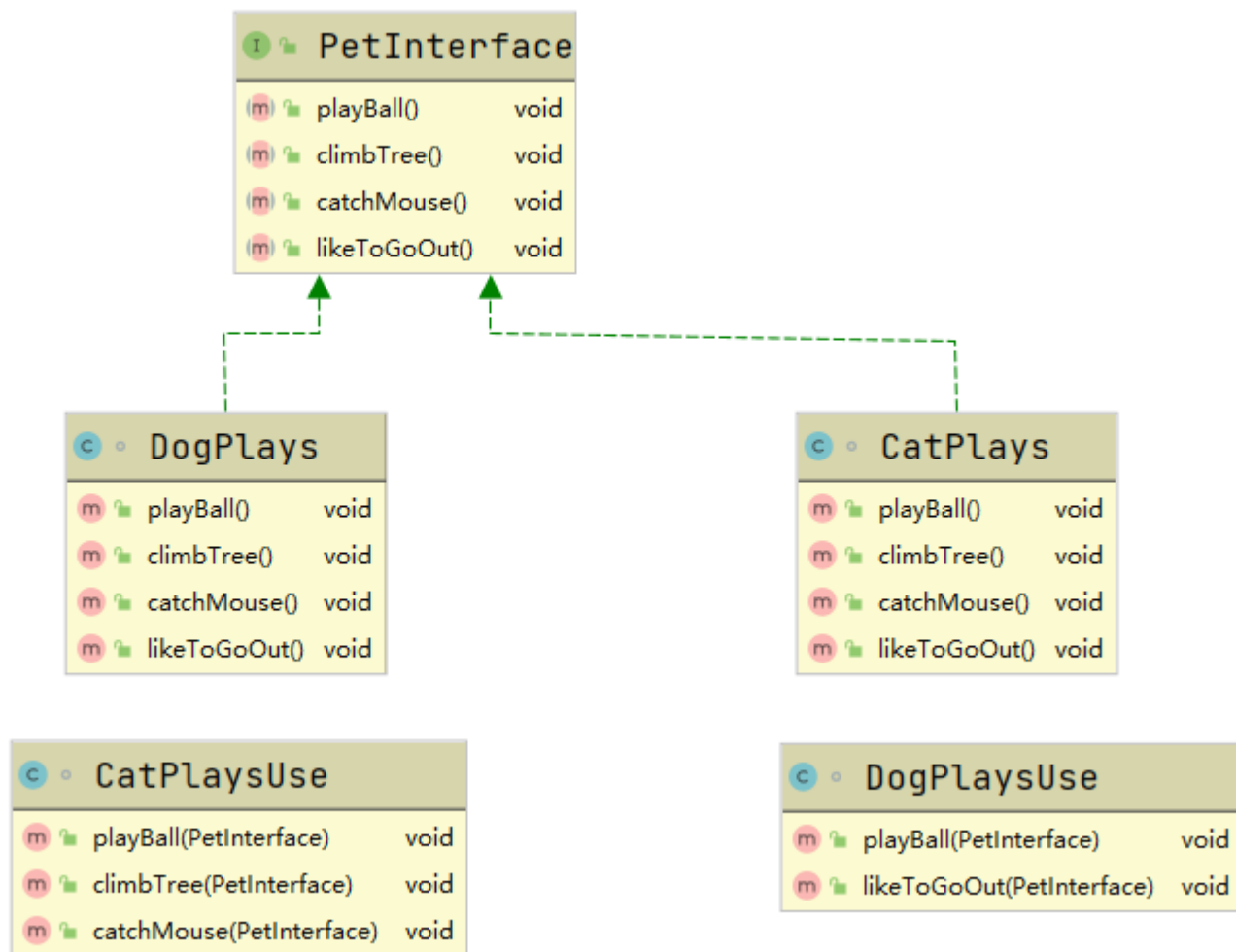
Pillar类不再依赖具体类，因此每当系统增加新的Geometry的子类时，**比如增加一个Triangle子类，那么不需要修改Pillar类的任何代码，就可以使用Pillar创建出具有三角形底的柱体。**

- **场景1:** 当我们需要用其它组件完成任务时，只需要替换该接口的实现，代码的其它部分不需要改变。
- **场景2:** 当现有的组件不能满足要求时，我们可以创建新的组件，实现该接口，或者，直接对现有的组件进行扩展，由子类去完成扩展的功能。
- **减少了代码耦合，实现了代码复用**

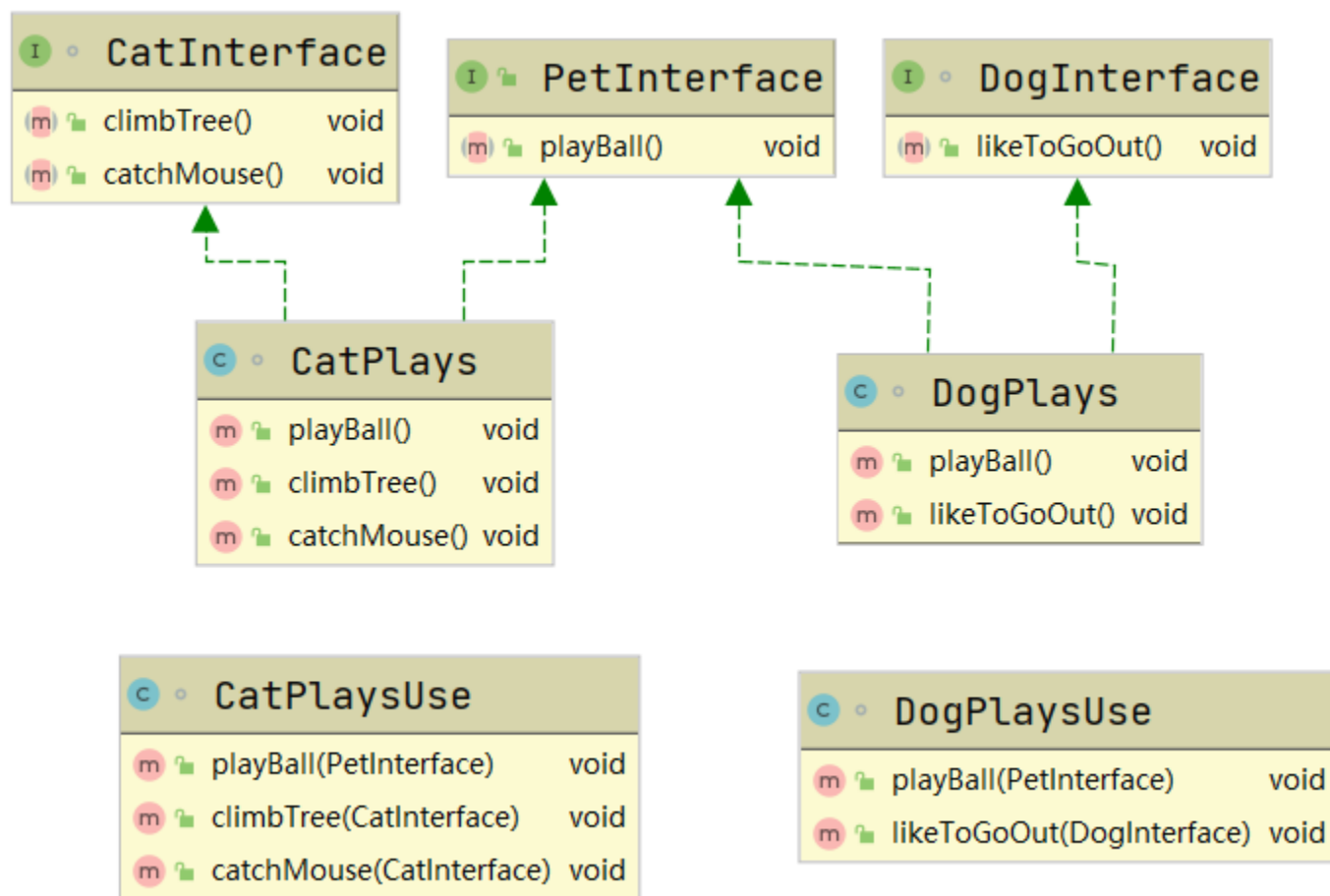
接口隔离原则

- **接口隔离原则**: 使用多个专门的接口, 而不使用单一的总接口, 即客户端不应该依赖那些它不需要的接口。

违背接口隔离原则



接口隔离原则正确示范



<https://blog.csdn.net/suixinfeixiangfei/article/details/122899213>

迪米特法则

- **迪米特法则**: 一个软件实体应当尽可能少地与其他实体发生相互作用。
- 迪米特法则又叫**最少知道原则**, 即一个类对自己依赖的类知道的越少越好。
- 对于被依赖的类不管多么复杂, 都尽量将逻辑封装在类的内部。对外除了提供public方法, 不对外泄露任何信息

违反迪米特法则

- 学校员工
- 学校管理员
 - 管理全校员工信息
- 学院员工
- 学院管理员
 - 管理学院员工信息

```
class Employee {  
    private String id;  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
}
```

```
class CollegeEmployee {  
    private String id;  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
}
```

```
class CollegeManager{  
    //返回学院的所有员工  
    public List<CollegeEmployee> getAllEmployee(){  
        List<CollegeEmployee> list = new ArrayList<CollegeEmployee>();  
        //增加了10个员工到list  
        for (int i = 0; i < 10; i++) {  
            CollegeEmployee emp = new CollegeEmployee();  
            emp.setId("学院员工ID="+i);  
            list.add(emp);  
        }  
        return list;  
    }  
}
```

只与直接的朋友通信

https://blog.csdn.net/GXL_1012/article/details/108756528

违反迪米特法则

- 学校员工
- 学校管理员
 - 管理全校员工信息
- 学院员工
- 学院管理员
 - 管理学院员工信息

只与直接的朋友通信

https://blog.csdn.net/GXL_1012

2024/10/31

```
class SchoolManager{
    //返回学校总部的员工
    public List<Employee> getAllEmployee(){
        List<Employee>list = new ArrayList<Employee>();
        for (int i = 0; i < 5; i++) {
            Employee emp = new Employee();
            emp.setId("学校总部员工ID="+i);
            list.add(emp);
        }
        return list;
    }
    //该方法完成输出学校总部和学院员工信息 (ID)
    void PRINTAllEmployee(CollegeManager sub){
        //分析问题
        //1、这两的CollegeEmployee不是SchoolManager的直接朋友
        //2、CollegeEmployee是以局部变量方式出现在SchoolManager
        //3、违反了迪米特法则
        //获取学院员工
        List<CollegeEmployee>list1 = sub.getAllEmployee();
        System.out.println("=====学院员工=====");
        for (CollegeEmployee e :list1) {
            System.out.println(e.getId());
        }
        //获取学院总部员工
        List<Employee>list2 = this.getAllEmployee();
        System.out.println("=====学院总部员工=====");
        for (Employee e :list2) {
            System.out.println(e.getId());
        }
    }
}
```



迪米特法则正确示范

```
class CollegeManager{
    //返回学院的所有员工
    public List<CollegeEmployee> getAllEmployee(){
        List<CollegeEmployee> list = new ArrayList<CollegeEmployee>();
        //增加了10个员工到list
        for (int i = 0; i < 10; i++) {
            CollegeEmployee emp = new CollegeEmployee();
            emp.setId("学院员工ID="+i);
            list.add(emp);
        }
        return list;
    }
    //输出学院员工的信息
    public void printEmployee(){
        //获取到学院员工
        List<CollegeEmployee>list1 = getAllEmployee();
        System.out.println("=====学院员工=====");
        for (CollegeEmployee e :list1) {
            System.out.println(e.getId());
        }
    }
}
```

```
class SchoolManager{
    //返回学校总部的员工
    public List<Employee> getAllEmployee(){
        List<Employee>list = new ArrayList<Employee>();
        for (int i = 0; i < 5; i++) {
            Employee emp = new Employee();
            emp.setId("学校总部员工ID="+i);
            list.add(emp);
        }
        return list;
    }
    //该方法完成输出学校总部和学院员工信息 (ID)
    void printAllEmployee(CollegeManager sub){
        //分析问题
        //1. 将输出学院员工方法, 封装到CollegeManager
        sub.printEmployee();

        //获取学院总部员工
        List<Employee>list2 = this.getAllEmployee();
        System.out.println("=====学院总部员工=====");
        for (Employee e :list2) {
            System.out.println(e.getId());
        }
    }
}
```

小结 - 软件设计七大原则

- **开闭原则**: 一个软件实体应当对扩展开放, 对修改关闭。
- **单一职责原则**: 一个类只负责一个功能领域中的相应职责。
- **里氏代换原则**: 所有引用基类（父类）的地方必须能透明地使用其子类的对象。
- **合成复用原则**: 尽量使用组合或者聚合关系实现代码复用, 少使用继承。

小结 - 软件设计七大原则

- **依赖倒转原则**: 抽象不应该依赖于细节, 细节应当依赖于抽象。换言之, 要针对接口编程, 而不是针对实现编程。
- **接口隔离原则**: 使用多个专门的接口, 而不使用单一的总接口, 即客户端不应该依赖那些它不需要的接口。
- **迪米特法则**: 一个软件实体应当尽可能少地与其他实体发生相互作用。

小结:

- 在程序设计时，我们应该将程序功能最小化，每个类只干一件事。
- 若在类似功能基础之上添加新功能，则要合理使用继承。
- 对于多方法的调用，要会运用接口，同时合理设置接口功能与数量。
- 最后类与类之间做到低耦合高内聚。

记忆口诀：访问加限制，函数要节俭，依赖不允许，动态加接口，父类要抽象，扩展不更改。

主要内容

- 面向对象设计的原则
- 对象的equals()方法与操作符“==”
- 内部类
- 匿名类
- 内部类的用途

Java类的祖先类:Object类

- 所有的Java类都直接或间接地继承了 `java.lang.Object` 类， `Object` 类是所有Java类的祖先，在这个类中定义了所有的Java对象都具有的方法。
 - `toString()`: 返回当前对象的字符串表示。
 - `equals (Object obj)`: 比较两个对象是否相等。仅当被比较的**两个引用变量指向同一对象时**， `equals()` 方法返回 `true` (判断是否同一引用)。

对象的equals()方法与操作符“==”

- equals()方法是在Object类中定义的方法，它的声明格式如下：

`public boolean equals(Object obj)`

- Object类的equals()方法的比较规则为：当参数obj引用的对象与当前对象为同一个对象，就返回true，否则返回false：

```
public boolean equals(Object obj){  
    if(this==obj)  
        return true;  
    else  
        return false;  
}
```

对象的equals()方法与操作符“==”

- 当操作符“==”两边都是引用类型变量时，这两个引用变量必须都引用同一个对象，结果才为true。

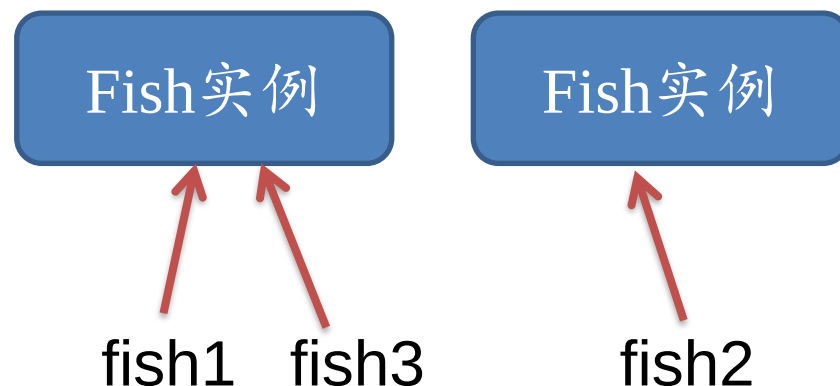
```
Food fish1=new Fish();
```

```
Food fish2=new Fish();
```

```
Food fish3=fish1;
```

```
System.out.println(fish1 ==fish2); //打印false
```

```
System.out.println(fish1==fish3); //打印true
```



对象的equals()方法与操作符“==”

- Object类中：
 - 判断是否为同一引用可以使用：对象的equals()方法与操作符“==”，他们是等价的。
- 某些类例外，对象的equals()方法与操作符“==”不等价

对象的equals()方法与操作符“==”

- 在JDK中有一些类覆盖了Object类的equals()方法，它们的比较规则为：**如果两个对象的类型一致，并且内容一致，则返回true。**
- 这些类包括：
 - **java.io.File、**
 - **java.util.Date、**
 - **java.lang.String、**
 - **包装类（如java.lang.Integer和java.lang.Double类等）。**

对象的equals()方法与操作符“==”

```
String str1=new String("Hello");
```

```
String str2=new String("Hello");
```

```
System.out.println(str1==str2); //打印false
```

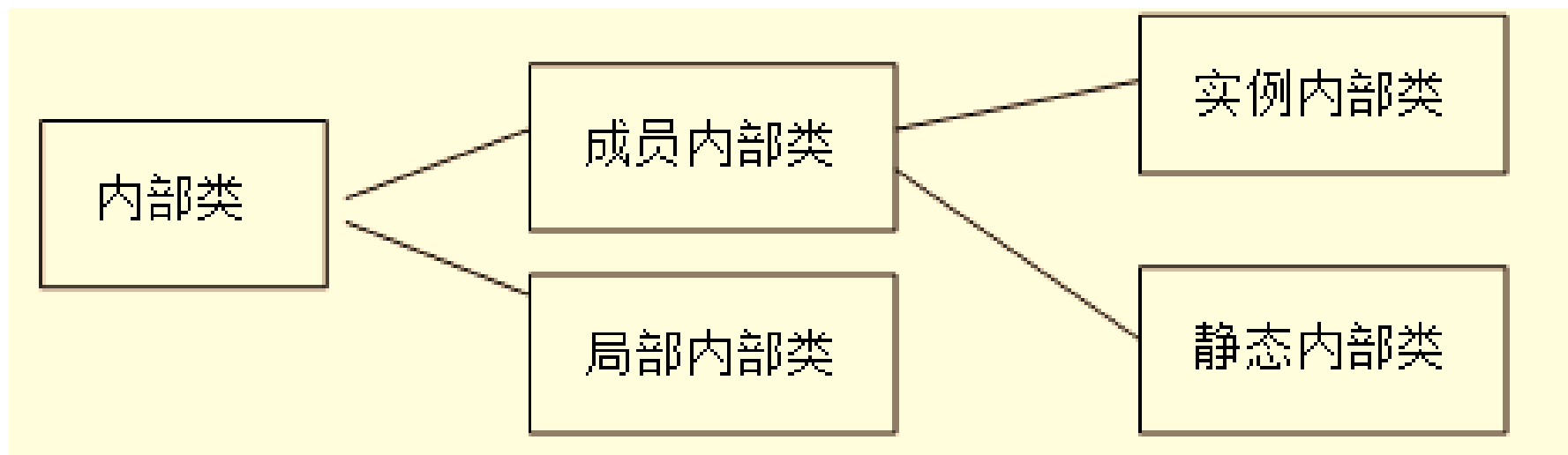
```
System.out.println(str1.equals(str2)); //打印true
```



主要内容

- 面向对象设计的原则
- 对象的equals()方法与操作符“==”
- 内部类
- 匿名类
- 内部类的用途

内部类的分类



示例代码

```
class Outer {  
    public class InnerTool { // 内部类  
        public int add(int a, int b) {  
            return a + b;  
        }  
    }  
    private InnerTool tool = new InnerTool();  
  
    public void add(int a, int b, int c) {  
        tool.add(tool.add(a, b), c);  
        System.out.println(tool.add(tool.add(a, b), c));  
    }  
}
```

示例代码

```
public class Tester {  
    public static void main(String args[]) {  
        Outer o = new Outer();  
        o.add(1, 2, 3);  
        Outer.InnerTool tool = new Outer().new InnerTool();  
    }  
}
```

在创建实例内部类的实例时，外部类的实例
必须已经存在

创建实例内部类的实例

- 在创建实例内部类的实例时，外部类的实例必须已经存在
 - 例如：要创建InnerTool类的实例，必须先创建Outer外部类的实例
 - **Outer.InnerTool tool=new Outer().new InnerTool();**
 - 以上代码等价于：
 - **Outer outer=new Outer();**
 - **Outer.InnerTool tool =outer.new InnerTool();**

实例内部类访问外部类的成员

- 在内部类中，可以直接访问外部类的所有成员，包括成员变量和成员方法。
- 实例内部类的实例自动持有外部类的实例的引用。



示例代码

内部类中，可以直接访问外部类的所有成员，包括成员变量和成员方法

```
public class A {  
    private int a1;  
    public int a2;  
    static int a3;  
    public A(int a1, int a2) {  
        this.a1 = a1;  
        this.a2 = a2;  
    }  
    protected int methodA() {  
        return a1 * a2;  
    }  
}
```

```
class B{ //内部类  
    int b1=a1; //直接访问private的a1  
    int b2=a2; //直接访问public的a2  
    int b3=a3; //直接访问static的a3  
    int b4=new A(3,4).a1; //访问一个新建的实例A的a1  
    int b5=methodA(); //访问methodA()方法  
}
```

示例代码

```
public static void main(String args[]){  
    A.B b=new A(1,2).new B();  
    System.out.println("b.b1="+b.b1);    //打印b.b1=1  
    System.out.println("b.b2="+b.b2);    //打印b.b2=2  
    System.out.println("b.b3="+b.b3);    //打印b.b3=0  
    System.out.println("b.b4="+b.b4);    //打印b.b4=3  
    System.out.println("b.b5="+b.b5);    //打印b.b5=2  
}  
}
```


静态内部类

- 静态内部类的实例不会自动持有外部类的特定实例的引用，在创建内部类的实例时，不必创建外部类的实例。

示例代码

- 例如以下类A有一个静态内部类B，客户类Tester创建类B的实例时不必创建类A的实例：

```
class AA {  
    public static class B {  
        int v;  
    }  
}  
  
class Tester {  
    public void test() {  
        AA.B b = new AA.B();  
        b.v = 1;  
    }  
}
```

静态内部类

- 客户类可以通过完整的类名直接访问静态内部类的静态成员。

```
public class AAA {  
    public static class B{  
        int v1;  
        static int v2;  
        public static class C{  
            static int v3;  
            int v4;  
        }  
    }  
}
```

示例代码

```
public class TesterAAA {  
    public void test() {  
        AAA.B b = new AAA.B();  
        AAA.B.C c = new AAA.B.C();  
        b.v1 = 1;  
        b.v2 = 1;  
        // AAA.B.v1=1; //编译错误  
        AAA.B.v2 = 1; // 合法  
        AAA.B.C.v3 = 1; // 合法  
    }  
}
```

局部内部类

- 局部内部类只能在当前方法中使用。
- 局部内部类和实例内部类一样，可以访问外部类的所有成员
- 此外，局部内部类还可以访问所在方法中的符合以下条件之一的参数和变量：
 - 最终变量或参数：用`final`修饰

局部内部类示例代码

```
public class AAAA {  
    int a;  
    public void method(final int p1, final int p2) {  
        final int localV1 = 1;  
        final int localV2 = 2;  
        int localV3 = 0;  
        localV3 = 1; // 修改局部变量  
        class B {  
            int b1 = a; // 合法, 访问外部类的实例变量  
            int b2 = p1; // 合法, 访问final类型的参数  
            int b3 = p2; // 合法, 访问final类型的参数  
            int b4 = localV1; // 合法, 访问实际上的最终变量  
            int b5 = localV2; // 合法, 访问最终变量  
            // int b6=localV3; //编译错误, localV3不是最终变量或者实际上的最终变量  
        }  
    }  
}
```

内部类封装类型

- 顶层类只能处于public和默认访问级别
- 而成员内部类可以处于**public**、**protected**、**默认**和**private**四个访问级别。



主要内容

- 面向对象设计的原则
- 对象的equals()方法与操作符“==”
- 内部类
- 匿名类
- 内部类的用途

匿名类

- 匿名类就是没有名字的类，是将类和类的方法定义在一个表达式范围里。
- 匿名类本身没有构造方法，但是会调用父类的构造方法。
- 匿名内部类将内部类的定义与生成实例的语句合在一起，并省去了类名以及关键字“class”，“extends”和“implements”等。

```
父类/接口 a = new 父类/接口(参数) {  
    方法……  
}
```

示例代码

```
public class AAAAA {  
    AAAAA(int v) {  
        System.out.println("another constructor");  
    }  
    AAAAA() {  
        System.out.println("default constructor");  
    }  
    void method() {  
        System.out.println("from AAAAA");  
    };  
    public static void main(String args[]) {  
        new AAAAA().method(); // 打印from AAAAA  
        AAAAA a = new AAAAA() { // 匿名类  
            void method() {  
                System.out.println("from anonymous");  
            }  
        };  
        a.method(); // 打印from anonymous  
    }  
}
```

default constructor
from AAAAA
default constructor
from anonymous

匿名类没有构造方法

```
public static void main(String args[]) {  
    int v = 1;  
    AAAAA a = new AAAAA(v) { // 匿名类  
        void method() {  
            System.out.println("from anonymous");  
        }  
    };  
    a.method(); // 打印from anonymous  
}
```

another constructor
from anonymous

主要内容

- 面向对象设计的原则
- 对象的equals()方法与操作符“==”
- 内部类
- 匿名类
- 内部类的用途

1. 封装类型

- 如果一个类只能有系统中的某一个类访问，可以定义为该类的内部类。
- 此外，如果一个内部类仅仅为特定的方法提供服务，那么可以把这个内部类定义在方法之内。
- 可见，**内部类是一种封装类型的有效手段。**

内部类封装类型

```
public interface Tool {  
    public int add(int a, int b);  
}
```

```
public class Outer {  
    private class InnerTool implements Tool {  
        public int add(int a, int b) {  
            return a + b;  
        }  
    }  
    public Tool getTool() {  
        return new InnerTool();  
    }  
    public static void main(String[] args){  
        //InnerTool实例向上转型为Tool类型  
        Tool tool=new Outer().getTool();  
        Tool tool1=new Outer().new InnerTool();  
    }  
}
```

内部类封装类型（续）

- 在客户类中不能访问Outer.InnerTool类，但是可以通过Outer类的getTool()方法获得InnerTool的实例

1. 封装类型

2. 直接访问外部类的成员

- 内部类的一个特点是能够访问外部类的各种访问级别的成员。
- 假定有类A和类B，类B的`reset()`方法负责重新设置类A的实例变量`count`的值。
 - 一种实现方式是把类A和类B都定义为外部类：

示例代码

```
class A{  
    private int count;  
    public int add(){return ++count;}  
  
    public int getCount(){return count;}  
  
    public void setCount(int count){  
        this.count=count;}  
}
```

```
class B{  
    A a; //类B与类A关联  
    B(A a){this.a=a;}  
    public void reset(){  
        if(a.getCount()>0)  
            a.setCount(1);  
        else  
            a.setCount(-1);  
    }  
}
```

内部类访问外部类的成员

- 假如需求中要求类A的count属性不允许被除类B以外的其他类读取或设置，那么以上实现方式就不能满足这一需求。
- 在这种情况下，把类B定义为内部类就可以解决这一问题，而且会使程序代码更加简洁

示例代码

```
class A{  
    private int count;  
    public int add(){return ++count;}
```

```
class B{ //定义内部类B  
    public void reset(){  
        if(count>0)  
            count=1;  
        else  
            count=-1;  
    }  
}
```

```
}
```

内部类的用途

1. 封装类型
2. 直接访问外部类的成员
3. 回调

回调

- 在以下Adjustable接口和Base类中都定义了adjust()方法，这两个方法的参数签名相同，但是有着不同的功能。

```
public interface Adjustable{
```

```
    /** 调节温度 */
```

```
    public void adjust(int temperature);
```

```
}
```

```
public class Base{
```

```
    private int speed;
```

```
    /** 调节速度 */
```

```
    public void adjust(int speed){
```

```
        this.speed=speed;
```

```
    }
```

```
}
```

回调

- 如果有一个Sub类同时具有调节温度和调节速度的功能，那么Sub类需要继承Base类，并且实现Adjustable接口，但是以下代码并不能满足这一需求：

adjust(int speed)
调节速度

```
public class Sub extends Base implements Adjustable{  
    private int temperature;  
    public void adjust(int temperature){  
        this.temperature=temperature;  
    }  
}
```

adjust(int temerature)
调节温度

回调

```
public class Sub extends Base {  
    private int temperature;
```

```
    private void adjustTemperature(int temperature){  
        this.temperature=temperature;  
    }
```

```
    private class Closure implements Adjustable{  
        public void adjust(int temperature){  
            adjustTemperature(temperature);  
        }  
    }
```

```
    public Adjustable getCallbackReference(){  
        return new Closure();  
    }
```

```
}
```

2024/10/31

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

- 以下代码演示客户类使用Sub类的调节温度的功能：

```
public class TestClass {  
    public static void main(String[] args){  
        //调节温度  
        Sub sub=new Sub();  
        Adjustable ad=sub.getCallBackReference();  
        ad.adjust(15);  
  
        //调节速度  
        sub.adjust(350);  
    }  
}
```

接口回调

回调

- 回调实质上是指一个类尽管实际上实现了某种功能,但是没有直接提供相应的接口,客户类可以通过这个类的内部类的接口来获得这种功能。而这个内部类本身并没有提供真正的实现,仅仅调用外部类的实现。
- 可见,回调充分发挥了内部类具有访问外部类的实现细节的优势。

```
public class Sub extends Base {  
    private int temperature;  
  
    private void adjustTemperature(int temperature){  
        this.temperature=temperature;  
    }  
  
    private class Closure implements Adjustable{  
        public void adjust(int temperature){  
            adjustTemperature(temperature);  
        }  
    }  
    public Adjustable getCallBackReference(){  
        return new Closure();  
    }  
}
```



内部类的文件

- 对于每个内部类，Java编译器会生成独立的.class文件。这些类文件的命名规则如下：
 - 成员内部类：外部类的名字\$内部类的名字
 - 局部内部类：外部类的名字\$数字和内部类的名字
 - 匿名类：外部类的名字\$数字



```

public class AAAAAA {
    static class B {
    } // 成员内部类, 对应A$B.class
    class C { // 成员内部类, 对应A$C.class
        class D {
        } // 成员内部类, 对应A$C$D.class
    }
    public void method1() {
        class E {
        } // 局部内部类1, 对应A$1E.class

        B b = new B() {
        }; // 匿名类1, 对应A$1.class
        C c = new C() {
        }; // 匿名类2, 对应A$2.class
    }
    public void method2() {
        class E {
        } // 局部内部类2, 对应A$2E.class
    }
}

```

Java编译器编译以上程序，
会生成以下类文件：

A.class

A\$B.class

A\$C.class

A\$C\$D.class

A\$1E.class

A\$1.class

A\$2.class

A\$2E.class

小结

比较方面	实例内部类	静态内部类	局部内部类
主要特征	内部类的实例引用特定的外部类的实例	内部类的实例不与外部类的任何实例关联	可见范围是所在的方法
可用的修饰符	访问控制修饰符， abstract,final	访问控制修饰符， static,abstract,final	abstract,final
可以访问外部类的哪些成员	可以直接访问外部类的所有成员	只能直接访问外部类的静态成员	可以直接访问外部类的所有成员，并且能访问所在方法的最终或实际上的最终变量和参数
拥有成员的类型	只能拥有实例成员	可以拥有静态成员和实例成员	只能拥有实例成员
外部类如何访问内部类的成员	必须通过内部类的实例来访问	对于静态成员，可以通过内部类的完整类名来访问	必须通过内部类的实例来访问

